

『校則問題』を再考する

—偏差値と校則の自由度に関する定量的分析—

大森一輝

一橋大学ソーシャル・データサイエンス学部 2 年
5125006H

2026 年 5 月 31 日

1 データ分析の目的・対象・方法

1.1 研究の目的と先行研究

本レポートの目的は、筆者が高校時代に実施した探究活動（大森、2023）を発展させ、「偏差値が高いほど校則が緩やかである」という仮説を定量的に再検証することである。

近年、理不尽な内容や過度な頭髪・服装指導を伴ういわゆる「ブラック校則」の見直しが社会的な議論を呼んでいる。しかし、校則の厳しさをめぐる議論の多くは、個別の事例や主観的な印象に基づく定性的なものにとどまっており、全国規模での実証的な分析は乏しい。先行研究（大森、2023）では、全国の公立高等学校 32 校を対象に、偏差値と校則の拘束度の関係を条文一つひとつを観点別に手作業で分類し、相関分析を行った。その結果、「偏差値が高い学校ほど校則が緩やかである」という傾向が示唆された。この傾向が実証的に確認されれば、学校の学力水準によって、生徒に許容される自律性や自由度に構造的な格差が存在する可能性を示し、教育社会学的に重要な意義を持つ。

一方で、先行研究には以下の 3 点の課題があった。

1. 手作業による校則のデータ収集に依存していたため、分析対象が 32 校に限定され、全国的な傾向を十分に捉えられなかった。
2. 校則の拘束度の評価が筆者の主観的な判断に依拠しており、客観性と再現性に課題があった。
3. 相関関係の観察にとどまり、統計的有意性の検証が行われていなかった。

以上を踏まえ、本研究では Web スクレイピングと形態素解析を活用し、分析対象を全国規模に拡大するとともに、ルールベースの自動アノテーションにより校則の自由度を客観的な指標として算出することで、仮説の統計的な再検証を試みる。

1.2 「自由度」の定義と本研究の立場

本研究では「校則の自由度」を、校則において生徒の行動を許容や裁量に委ねる表現（推奨・例外）が、禁止・義務表現に対してどの程度含まれるかとして定量化する。ただし、この指標には本質的な限界がある。校則が「自由」であるとは、必ずしも多くの許可条項が存在することを意味しない。むしろ本当に自由な学校は、そもそも規定を設ける必要がないために条文自体が存在しない可能性がある。つまり、本研究が測定しているのは「明文化された規範表現の中での自由化」であり、「規定されない自由」は観測できない。

1.3 分析の対象と変数の定義

本研究では、全国の公立高等学校を分析対象とする。主要な変数を以下のように定義する。

- **独立変数（学力水準）**：「高校偏差値.net」^{*1}より取得した各高等学校の偏差値^{*2}
- **統制変数（都市度）**：e-Stat（政府の総合窓口）より取得した市区町村別の DID 人口比率
- **統制変数（学校属性）**：男女別（女子校ダミー・男子校ダミー）、専門高校ダミー
- **従属変数（校則の自由度）**：「全国校則一覧」^{*3}に掲載された校則に対し、形態素解析 Mecab とルールベースのアノテーションを適用して算出する自由度スコア

これらの変数を用い、以下の2つのリサーチクエスチョン（RQ）を設定する。

RQ1：偏差値は、自由度と統計的に有意な関係を持つか。

RQ2：偏差値帯によって、校則の言語的構成（禁止・義務・推奨・例外）にどのような差異がみられるか。

1.4 分析の流れ

分析は以下の手順で実施した。

1. 校則データおよび偏差値データの Web スクレイピングによる収集
2. DID データの e-Stat API からの取得
3. テキスト前処理（定型文除去・文分割）
4. Mecab 形態素解析による頻出語の確認
5. ルールベースの自動アノテーション
6. 自由度スコアの算出
7. 偏差値データとの結合
8. 重回帰分析・偏差値帯別比較による統計的検証

なお、以降で述べる各工程の実装コードは量が多いため、本文では処理の目的・設計判断・主要なコード・結果を中心に記述し、詳細な解説は補足資料（Jupyter ノートブック）^{*4}に掲載した。

^{*1} 「高校偏差値.net」 <https://xn--swqwd788bm2jy17d.net/>

^{*2} 同サイトの偏差値は、統一模試等の数値とは異なり、「全国の塾関係者から集めたデータを元に算出」されている。独自調査によって算出されたデータであるため、偏差値データは合格難易度の目安（代表値）として解釈する。

^{*3} 「全国校則一覧」 <https://www.kousoku.org/>

^{*4} 補足資料：https://github.com/kazuki-ok1201/kousoku-analysis/blob/main/OMORI_KAZUKI_datahandling_report_

2 データ収集

2.1 データ収集方法の検討

2.1.1 校則データの検討

先行研究（大森、2023）を参考に「全国校則一覧」を利用することとした。同サイトは「情報公開条例に基づく情報公開請求によって開示された文書等に基づく非営利目的のウェブサイト」であり^{*5}、NPO 法人 Change of Perspective が運営している。情報公開請求によって開示された公立高校の校則を一元的に集約しているという公的な性質から、本研究の分析対象として適切であると判断した。

2.1.2 偏差値データの検討

当初は先行研究（大森、2023）でも参照した「みんなの高校情報」を使用する計画であった。しかし、同サイトの利用規約^{*6}を精査した結果、第 2 条第 1 項に「ロボット等機械的方法を用いて当社サイトのコンテンツを大量に複製、分析、参照する行為（スクレイピング等）をすること」が明示的に禁止されていることが判明した。同サイトを利用してスクレイピングを実施することはデータ倫理上の問題があるためと判断し、データソースを変更することとした。

代替として「高校偏差値.net」を選定した。同サイトは全国の塾関係者 25 名の意見交換会から生まれたサイトであり、偏差値は「全国の塾関係者から集めたデータを元に算出」されている。利用規約（第 2 条）^{*7}において、スクレイピングの明示的な禁止はなく、商業利用のみが禁止されていることが確認した。

2.1.3 DID データの検討

都市度の代理変数として、国勢調査の DID（人口集中地区）人口比率を採用した。DID 人口比率は市区町村の総人口に対する DID 人口の割合であり、都市化度合いを示す指標として用いることができる。データは e-Stat(政府統計の総合窓口) を API を用いて取得した。

2.2 データ倫理の確認

スクレイピングを API 利用を実施する前に、両サイトの利用規約や robots.txt を確認した。

全国校則一覧

robots.txt では、管理画面 (/wp-admin/) へのアクセスのみが Disallow に指定されており、各学校ページへのアクセスは許可されていた。免責事項^{*8}においても学術目的での分析を禁止する規定はなかった。

高校偏差値.net

robots.txt にアクセス制限の記述が存在せず、利用規約にもスクレイピングの禁止規定はなかった。商業

final.ipynb

^{*5} 全国校則一覧「当サイトについて」<https://www.kousoku.org/about/>

^{*6} みんなの高校情報「利用規約」<https://www.itokuro.jp/termsofservice/>

^{*7} 高校偏差値.net「利用規約」https://xn--swqwd788bm2jy17d.net/terms_and_agreements.php

^{*8} 全国校則一覧「免責事項」<https://www.kousoku.org/terms/>

利用のみが禁止されており、学術目的での利用は問題ないと判断した。

e-Stat（政府統計の総合窓口）

e-Stat の利用規約^{*9}では、公開されているコンテンツを「複製、公衆送信、翻訳・変形等の翻案等、自由に利用でき」とされており、商用利用も許可されている。出典の記載が求められているため、本レポートおよび補足ノートブックにおいて「出典：政府統計の総合窓口（e-Stat）」と明記した。なお、同規約は政府標準利用規約（第 2.0 版）に準拠しており、クリエイティブ・コモンズ・ライセンス（CC BY 4.0）と互換性がある。

以上の確認を踏まえ、全国校則一覧および高校偏差値.net へのリクエスト間に `time.sleep(2.0)` を設けてサーバー負に配慮した。e-Stat API については公式に提供されているインターフェースであり、通常の利用範囲内でリクエストを行った。

2.3 校則テキストの収集

Python の `requests` と `BeautifulSoup` を用いてスクレイピングを実施した。全国校則一覧は都道府県ページ（例：<https://www.kousoku.org/tokyo/>）の下に各学校の校則ページが配置されている構造になっている。まず都道府県ページから各学校の URL を取得し、次に各学校ページの HTML から学校名（h1 タグ）と校則本文（article タグなど）を抽出した。

ソースコード 1 校則テキスト収集

```
1 # 校則関数URL
2 def school_urls(pref, delay=1.5):
3     url = 'https://www.kousoku.org/' + pref + '/'
4     r = requests.get(url, timeout=10)
5     soup = BeautifulSoup(r.text, 'html.parser')
6     url_list = list()
7     for a in soup.find_all('a'):
8         href = a.get('href')
9         if href is None:
10             continue
11         base = 'https://www.kousoku.org/' + pref + '/d'
12         if base in href and href not in url_list:
13             url_list.append(href)
14     time.sleep(delay)
15     return url_list
16
17 # 校則スクレイピング関数
18 def scrape_school(url, delay=2.0):
19     try:
20         r = requests.get(url)
```

^{*9} e-Stat「利用規約」<https://www.e-stat.go.jp/terms-of-use>

```

21     soup = BeautifulSoup(r.text, 'html.parser')
22     h1 = soup.find('h1')
23     title = h1.text.strip() if h1 else ''
24     if '】' in title:
25         school_name = title.split('】')[1]
26     else:
27         school_name = title
28     if 'の校則' in school_name:
29         school_name = school_name.split('の校則')[0]
30     pref = url.split('/')[3]
31     text = soup.text
32     year = None
33     if '年度' in text:
34         idx = text.find('年度')
35         year_str = text[idx-4:idx]
36         is_number = True
37         for char in year_str:
38             if char not in '0123456789':
39                 is_number = False
40         if is_number:
41             year = int(year_str)
42     content = (
43         soup.find('article') or
44         soup.find('div', class_='entry-content')
45     )
46     if content:
47         raw_text = content.text
48     else:
49         raw_text = ''
50     time.sleep(delay)
51     return {
52         'school_name': school_name,
53         'prefecture': pref,
54         'year': year,
55         'url': url,
56         'raw_text': raw_text
57     }
58 except Exception as e:
59     print('エラー:', url)
60     time.sleep(delay)

```

```

61     return None
62
63 # 全都道府県のリスト
64 PREF_LIST = [
65     'hokkaido',
66     'aomori', 'iwate', 'miyagi', 'akita', 'yamagata', 'fukushima',
67     'ibaraki', 'tochigi', 'gunma', 'saitama', 'chiba', 'tokyo', 'kanagawa',
68     'niigata', 'toyama', 'ishikawa', 'fukui', 'yamanashi', 'nagano',
69     'shizuoka', 'aichi', 'gifu', 'mie',
70     'shiga', 'kyoto', 'osaka', 'hyogo', 'nara', 'wakayama',
71     'tottori', 'shimane', 'okayama', 'hiroshima', 'yamaguchi',
72     'tokushima', 'kagawa', 'ehime', 'kouchi',
73     'fukuoka', 'saga', 'nagasaki', 'kumamoto', 'oita',
74     'miyazaki', 'kagoshima', 'okinawa'
75 ]
76
77 # スクレイピング実行部分
78 records = list()
79 for pref in PREF_LIST:
80     print('\n', pref)
81     urls = school_urls(pref)
82     print('0-/', len(urls))
83     count = 0
84     for url in urls:
85         result = scrape_school(url)
86         if result:
87             records.append(result)
88         count = count + 1
89         if count % 20 == 0:
90             pd.DataFrame(records).to_csv(
91                 'school_rules.csv', index=False, encoding='utf-8-sig'
92             )
93             print(count, '/', len(urls), '保存済み')
94     print(count, '/', len(urls), '取得完了')

```

収集の結果、30 都道府県の公立高校 1,666 校分の校則テキストを取得した。なお、同サイトの掲載高校すうは全国の公立高校約 3,300 校のうち約半数であり、17 府県については掲載がなかった。データの網羅性には限界がある。

2.4 偏差値データの収集

同様に `requests` と `BeautifulSoup` を用いて、高校偏差値.net から全 47 都道府県の高校偏差値データを取得した。各都道府県ページの HTML テーブルから偏差値・学校名・設置者・男女別・学科・所在地を抽出した。

ソースコード 2 偏差値データ収集

```
1 # 偏差値取得関数
2 def get_hensachi(pref_key, pref_ja, delay=2.0):
3     url = 'https://xn--swqwd788bm2jy17d.net/' + pref_key + '.php'
4     try:
5         r = requests.get(url, timeout=15)
6         soup = BeautifulSoup(r.text, 'html.parser')
7         records = list()
8         for table in soup.find_all('table'):
9             for tr in table.find_all('tr'):
10                 tds = tr.find_all('td')
11                 if len(tds) < 2:
12                     continue
13                 hensachi_text = tds[0].text.strip()
14                 school_name = tds[1].text.strip()
15                 is_number = True
16                 for char in hensachi_text:
17                     if char not in '0123456789':
18                         is_number = False
19                 if not(is_number and len(hensachi_text) == 2):
20                     continue
21                 school_type = tds[2].text.strip()
22                 gender = tds[3].text.strip()
23                 gakka = tds[4].text.strip()
24                 location = tds[9].text.strip()
25                 records.append({
26                     'school_name': school_name,
27                     'hensachi': int(hensachi_text),
28                     'school_type': school_type,
29                     'gender': gender,
30                     'gakka': gakka,
31                     'location': location,
32                     'prefecture': pref_key,
33                     'prefecture_ja': pref_ja
```

```

34         })
35         time.sleep(delay)
36         return records
37     except Exception as e:
38         print('エラー:', pref_key)
39         time.sleep(delay)
40         return list()
41
42 # 全都道府県のローマ字と日本語のmap
43 PREF_MAP = [
44     ('hokkaido', '北海道'), ('aomori', '青森県'), ('iwate', '岩手県'),
45     ('miyagi', '宮城県'), ('akita', '秋田県'), ('yamagata', '山形県'),
46     ('fukushima', '福島県'), ('ibaraki', '茨城県'), ('tochigi', '栃木県'),
47     ('gunma', '群馬県'), ('saitama', '埼玉県'), ('chiba', '千葉県'),
48     ('tokyo', '東京都'), ('kanagawa', '神奈川県'), ('niigata', '新潟県'),
49     ('toyama', '富山県'), ('ishikawa', '石川県'), ('fukui', '福井県'),
50     ('yamanashi', '山梨県'), ('nagano', '長野県'), ('gifu', '岐阜県'),
51     ('shizuoka', '静岡県'), ('aichi', '愛知県'), ('mie', '三重県'),
52     ('shiga', '滋賀県'), ('kyoto', '京都府'), ('osaka', '大阪府'),
53     ('hyogo', '兵庫県'), ('nara', '奈良県'), ('wakayama', '和歌山県'),
54     ('tottori', '鳥取県'), ('shimane', '島根県'), ('okayama', '岡山県'),
55     ('hiroshima', '広島県'), ('yamaguchi', '山口県'), ('tokushima', '徳島県'),
56     ('kagawa', '香川県'), ('ehime', '愛媛県'), ('kouchi', '高知県'),
57     ('fukuoka', '福岡県'), ('saga', '佐賀県'), ('nagasaki', '長崎県'),
58     ('kumamoto', '熊本県'), ('oita', '大分県'), ('miyazaki', '宮崎県'),
59     ('kagoshima', '鹿児島県'), ('okinawa', '沖縄県'),
60 ]
61
62 # 都道府県立高校に含まれる文字列
63 PUBLIC_KEYWORDS = [
64     '道立', '都立', '府立', '県立',
65 ]
66
67 # 都道府県立判別
68 def is_public(school_type):
69     for keyword in PUBLIC_KEYWORDS:
70         if keyword in school_type:
71             return True
72     return False
73

```



```

74 # 学科判別
75 def aggregate_gakka(gakka_series):
76     for g in gakka_series:
77         if '普通' in str(g) or '総合' in str(g):
78             return '普通'
79     return gakka_series.iloc[0]
80
81 # 男女別判別
82 def aggregate_gender(gender_series):
83     genders = set()
84     for g in gender_series:
85         g = str(g)
86         if '/' in g:
87             return '共学'
88         genders.add(g)
89     if len(genders) == 1:
90         return genders.pop()
91     return '共学'
92
93 # スクレイピング実行
94 all_records = list()
95 for pref_key, pref_ja in PREFMAP:
96     records = get_hensachi(pref_key, pref_ja)
97     if records:
98         all_records.extend(records)
99         print(pref_ja, ': ', len(records), '件')
100     else:
101         print(pref_ja, ': 件0')
102
103 df_hensachi_all = pd.DataFrame(all_records)
104
105 # 都道府県立高校に絞る
106 df_hensachi = df_hensachi_all.loc[df_hensachi_all['school_type']]
107 .apply(is_public), :].copy()
108 df_hensachi.to_csv('hensachi.csv', index=False, encoding='utf-8-sig')

```

取得後、設置者が都道府県立の高校のみに絞り込み、6,278 件の偏差値レコードを得た。同一学校に複数の学科がある場合は偏差値の最大値を代表値として採用し、学科は普通科・総合学科が含まれれば「普通」に集約した。これは、全国校則一覧の校則が学科単位ではなく学校単位で記録されているため、偏差値も学校単位の代表値に統一しつつ必要があったためである。

2.5 DID データの取得

e-Stat API を用いて、令和 2 年度国勢調査から市区町村別の総人口（統計表 ID：0003445078）および DID 人口（統計表 ID：0003445079）を取得した。市区町村コードをキーに両データを結合し、以下の式で DID 人口比率を算出した。

$$\text{DID 人口比率} = \frac{\text{DID 人口}}{\text{総人口}} \times 100 \quad (1)$$

ソースコード 3 DID データ収集

```
1 APP_ID = '読者のアプリケーションID'
2 TOTAL_STATS_ID = '0003445078'
3 DID_STATS_ID = '0003445079'
4
5 def fetch_values(stats_id, app_id):
6     url = 'https://api.e-stat.go.jp/rest/3.0/app/json/getStatsData'
7     r = requests.get(
8         url,
9         params={
10             'appId': app_id,
11             'statsDataId': stats_id,
12             'metaGetFlg': 'N',
13             'cntGetFlg': 'N',
14             'limit': 100000
15         },
16         timeout=60
17     )
18     return r.json()[ 'GET_STATS_DATA' ][ 'STATISTICAL_DATA' ]
19         [ 'DATA_INF' ][ 'VALUE' ]
20
21 def fetch_area_map(stats_id, app_id):
22     url = 'https://api.e-stat.go.jp/rest/3.0/app/json/getMetaInfo'
23     r = requests.get(
24         url,
25         params={
26             'appId': app_id,
27             'statsDataId': stats_id
28         },
29         timeout=30
30     )
```

```

31     objs = r.json()[ 'GET_META_INFO' ][ 'METADATA_INF' ]
32     [ 'CLASS_INF' ][ 'CLASS_OBJ' ]
33     area_map = {}
34     for obj in objs:
35         if obj[ '@id' ] == 'area':
36             for c in obj[ 'CLASS' ]:
37                 # 市区町村のみ
38                 if c.get( '@level' ) in ( '4', '5', '6' ):
39                     area_map[ c[ '@code' ] ] = c[ '@name' ]
40     return area_map
41
42 def build_population_df( values ):
43     rows = []
44     for v in values:
45         area_code = v.get( '@area' )
46         # が無いものは除外 area
47         if area_code is None:
48             continue
49         try:
50             pop = int( v[ '$' ] )
51         except:
52             continue
53         rows.append( {
54             'area_code': area_code ,
55             'pop': pop
56         } )
57     return pd.DataFrame( rows )
58
59 # 総人口・人口をそれぞれ取得する DID
60 total_values = fetch_values( TOTAL_STATS_ID, APP_ID )
61 did_values   = fetch_values( DID_STATS_ID,   APP_ID )
62 df_total = build_population_df( total_values )
63 .rename( columns={ 'pop': 'total_pop' } )
64 df_did   = build_population_df( did_values )
65 .rename( columns={ 'pop': 'did_pop' } )
66
67 # 重複を除去した上でコードをキーに結合する
68 df_total = df_total.drop_duplicates( subset='area_code' )
69 df_did    = df_did.drop_duplicates( subset='area_code' )
70 df_ratio = pd.merge( df_total, df_did, on='area_code', how='inner' )

```

```

71
72 # 市区町村名を付与する
73 area_map = fetch_area_map(TOTAL_STATS_ID, APP_ID)
74 df_ratio['area_name'] = df_ratio['area_code'].map(area_map)
75 df_ratio = df_ratio.loc[df_ratio['area_name'].notna()].copy()
76
77 # 人口比率 $DID = \text{人口}DID \div \text{総人口} \times 100$ 
78 df_ratio['did_ratio'] = round(
79     df_ratio['did_pop'] / df_ratio['total_pop'] * 100, 2
80 )
81
82 df_ratio.to_csv('df_ratio.csv', index=False, encoding='utf-8-sig')
83
84 # 所在地とのマッチング
85 hensachi_did = pd.merge(
86     df_hensachi,
87     df_ratio,
88     left_on='location',
89     right_on='area_name',
90     how='left'
91 )
92 hensachi_did['did_ratio'] = hensachi_did['did_ratio'].fillna(0)
93 hensachi_did = hensachi_did.drop(columns=['area_name'])
94 hensachi_did.to_csv('hensachi_did.csv', index=False)

```

メタデータから市区町村コード → 市区町村名の対応表を作成し、高校偏差値.net の所在地列と照合して DID 人口比率を付与した。マッチしなかった約 20% の学校については、大半が DID 地区そのものが存在しない農村型の市町村であり、DID 人口比率 = 0 で正しい値であるため、欠損値を 0 で補完した。

3 データ加工

3.1 テキスト前処理

スクレイピングで取得したテキストには校則本文以外の文字列（SNS シェアボタン・フッタリンク等）が混在している。全校のテキストを行単位に分割し、出現回数の多い行を集計することで定型文を特定した。「シェアする」が全校のテキスト末尾に安定して出現することが判明したため、この行以降をカットポイントとして除去した。また、校則テキストには全角スペース（\u3000）やタブ文字が混在しており、文分割やパターンマッチに支障をきたすため、半角スペースに統一した。次に、クリーニング済みのテキストを句点で分割し、条文単位のリストに変換した。これは後続のアノテーション処理が文単位でラベルを付与する設計であるためである。

```
1 # 定型文の特定
2 line_count_dict = dict()
3 for text in df_schools['raw_text']:
4     seen = set()
5     for line in str(text).split('\n'):
6         line = line.strip()
7         if len(line) == 0:
8             continue
9         if line not in seen:
10             seen.add(line)
11             if line in line_count_dict:
12                 line_count_dict[line] += 1
13             else:
14                 line_count_dict[line] = 1
15 line_series = pd.Series(line_count_dict)
16 top_lines = line_series.sort_values(ascending=False).head(100)
17 for line, count in top_lines.items():
18     print(str(count), '校', line[:80])
19
20 # クリーニング関数
21 def clean_text(raw):
22     text = str(raw)
23     if 'シェアする' in text:
24         text = text.split('シェアする')[0]
25     text = text.replace('\u3000', ' ')
26     text = text.replace('\t', ' ')
27     return text.strip()
28
29 # 文単位への分割
30 def split_sentences(text):
31     text = text.replace('。', '。 \n')
32     text = text.replace('.', '. \n')
33     sentences = list()
34     for line in text.split('\n'):
35         line = line.strip()
36         if line != '':
37             sentences.append(line)
38     return sentences
```

```

39
40 # 適用
41 df_schools['clean_text'] = df_schools['raw_text'].apply(clean_text)
42 df_schools['sentences'] = df_schools['clean_text'].apply(split_sentences)
43 df_schools['sentence_count'] = df_schools['sentences'].apply(len)
44 df_schools['char_count'] = df_schools['clean_text'].apply(len)
45 df_schools.head()

```

3.2 MeCab 形態素解析

収集データが校則として適切に取得できているかを確認するため、形態素解析エンジン MeCab（辞書：unidic-lite）を用いて名詞・動詞・形容詞を抽出し、全校テキストの頻出語を集計した。その結果を図 1 に示す。「場合」「生徒」「着用」「禁止」などが上位に並び、校則として妥当なテキストが収集できていることが確認された。なお、unidic-lite の出力は、タブ区切りの第 3 項が基本形、第 4 項が品詞情報に対応している。

ソースコード 5 形態素分析

```

1 # 形態素分析
2 tagger = MeCab.Tagger()
3 STOPWORDS = {
4     'する', 'ある', 'いる', 'なる', 'もの', 'こと',
5     'これ', 'その', 'この', 'よう', 'ため', 'とき',
6     '有る', '在る', '居る', '為る', '成る', '成す',
7 }
8 def get_content_words(text):
9     result = tagger.parse(text)
10    words = list()
11    for line in result.split('\n'):
12        if line in ('EOS', '') or '\t' not in line:
13            continue
14        parts = line.split('\t')
15        # parts[3] = 基本形、parts[4].split('-')[0] = 品詞の大分類
16        base = parts[3]
17        pos = parts[4].split('-')[0]
18        if pos in ('名詞', '動詞', '形容詞'):
19            if len(base) > 1 and base not in STOPWORDS:
20                has_digit = False
21                for char in base:
22                    if char in '01234567890123456789':
23                        has_digit = True
24                if not has_digit:

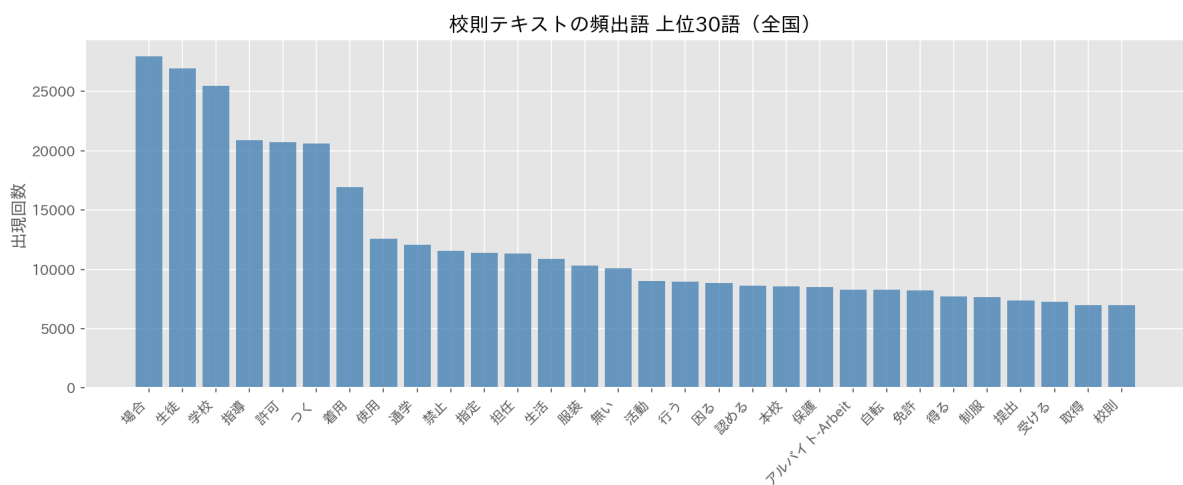
```

```

25         words.append(base)
26     return words
27
28 # カウント関数
29 word_count_dict = dict()
30 for text in df_schools['clean_text']:
31     for word in get_content_words(text):
32         if word in word_count_dict:
33             word_count_dict[word] += 1
34         else:
35             word_count_dict[word] = 1
36
37 # 可視化
38 word_series = pd.Series(word_count_dict)
39 top30 = word_series.sort_values(ascending=False).head(30)
40 fig, ax = plt.subplots(figsize=(12, 5))
41 ax.bar(range(len(top30)), top30.values, color='steelblue', alpha=0.8)
42 ax.set_xticks(range(len(top30)))
43 ax.set_xticklabels(top30.index.tolist(), rotation=45, ha='right')
44 ax.set_title('校則テキストの頻出語・上位語（全国）30')
45 ax.set_ylabel('出現回数')
46 plt.tight_layout()
47 plt.show()

```

図 1



3.3 自動アノテーション

3.3.1 ラベル体系の設計

各条文に P（禁止）・O（義務）・R（推奨）・E（例外）・A（曖昧）・OTHER（その他）の 6 ラベルのいずれかを付与する。各ラベルは単語の表層形ではなく、その表現が生徒をどのように制約・方向づけるかという統制的機能の観点から定義した（表 1）。

表 1 アノテーションラベルの定義と代表的表現

ラベル	定義	代表的表現
P（禁止）	生徒の行為を直接制限する	「禁止する」「してはならない」
O（義務）	特定行動を義務として要求	「～すること」「着用する」
R（推奨）	努力目標・望ましい状態	「心がける」「望ましい」
E（例外）	裁量・逸脱可能性を認める	「ただし」「を認める」
A（曖昧）	解釈を学校側に委ねる規範	「高校生らしい」「華美」
OTHER（その他）	上記いずれにも該当しない	理念・前文・手続説明等

3.3.2 パターンリストの構築

パターンリスト以下の手順で構築した。まず各ラベルの候補表現を列挙し、全文（約 22 万件）に対する出現件数を集計した。出現頻度が高い表現ほど校則テキストにおいて安定して使われていると考えられるため、この頻度を根拠として採用・除外を判断した。

ソースコード 6 パターンリストの構築

```
1 # 各ラベルの候補表現の出現数を集計する
2 candidate_patterns = {
3     'P': [
4         '禁止', 'してはならない', 'しないこと', '禁ずる', '禁じる', 'してはいけない',
5         '慎む', '許可なく', '無断で', '厳禁', '控えること', '認めない', '不可',
6         'いけない', '行ってはならない', '持ち込まない',
7         '没収', '預かる', '指導の対象', '特別指導',
8     ],
9     'O': [
10        'すること', 'しなければならない', 'でなければならない', '遵守', '着用する',
11        '従うこと', 'するものとする', 'を守ること', 'してください', 'するように',
12        'しなくてはならない', '提出すること', '届け出ること', '申し出ること',
13        '着用すること', '連絡すること', '行うこと', '注意すること', '指定の', '所定の',
14        '厳守', '必ず', 'とする', '義務',
15    ],
16     'R': [
```



```

17         '心がける', '努める', '望ましい', 'が好ましい',
18         'に努力', '励む', 'よう努める', 'を心がけ', '推奨', '基本とする',
19         'に心がける', 'するよう心がけ', 'することが大切', '求められる',
20     ],
21     'E': [
22         'ただし', 'やむを得ない', 'この限りではない', '許可を得',
23         'してもよい', 'することができる', 'を除く', '場合に限り',
24         '任意', '自由とする', 'してよい', '認める', 'でもよい',
25         '可とする', '許可される', '場合はこの限り', '承認を得た',
26         '差し支えない', 'してもかまわない', 'この限りでない',
27         '異装', '特例', '特別に', '例外'
28     ],
29     'A': [
30         '高校生らしい', '高校生として', '生徒として', '華美', '節度', '品位', '奇抜',
31         'ふさわしい', '常識', '清潔', '端正', '適切', '生徒らしい', '良識', '自然な',
32         'にふさわしい', 'にふさわしくない', '高校生らしく', 'マナー', '適切',
33         '節度ある', '相応しい', '相応しくない', '社会通念', '見苦しくない',
34     ]
35 }
36
37 all_sentences = list()
38 for sentences in df_schools['sentences']:
39     all_sentences.extend(sentences)
40 for label, patterns in candidate_patterns.items():
41     pat_count = dict()
42     for pat in patterns:
43         count = 0
44         for sent in all_sentences:
45             if pat in sent:
46                 count += 1
47         pat_count[pat] = count
48     sorted_pats = sorted(pat_count.items(),
49                           key = lambda x: x[1], reverse=True)
50     for pat, count in sorted_pats:
51         print(str(count), '件', label, pat)

```

たとえば「不可」は「不可欠」「不可能」への誤って検出するリスクがあるため「不可とする」「は不可」の形で採用した。「とする」はOラベルの最多候補だが、「本規定は○○とする」のような定義文に頻出するためOTHERとの競合が大きく、除外した。

ソースコード 7 アノテーションの実装

```

1 OTHER_PATS = [
2     '目指し', 'の精神に則', 'に貢献', 'の自覚を持', '人間として',
3     'を目的とする', 'の向上を図', '本校生徒は', 'を培う',
4 ]
5
6 P_PATS = [
7     '禁止', 'してはならない', 'しないこと',
8     'を禁ずる', 'してはいけない', '慎む',
9     '行ってはならない', '許可なく', '無断で',
10    '厳禁', '控えること', '認めない',
11    '不可とする', 'は不可', '許可しない',
12    'いけない', 'を持ち込まない', '持ち込み禁止',
13 ]
14
15 O_PATS = [
16    'すること', 'しなければならない', 'でなければならない',
17    'を遵守', '着用する', '従うこと',
18    'するものとする', 'しなくてはならない', '届け出ること',
19    '申し出ること', '提出すること', 'を着用すること',
20    '連絡すること', 'を行うこと', '注意すること',
21 ]
22
23 R_PATS = [
24    '心がける', '努める', '望ましい',
25    'が好ましい', 'に努力', '励む',
26    'よう心がける', 'を心がけ', 'に心がける',
27    'するよう心がけ', 'することが大切',
28 ]
29
30 E_PATS = [
31    'ただし', 'やむを得ない', 'この限りでない',
32    'この限りではない', '許可を得', 'してもよい',
33    'することができる', 'を除く', '場合に限り',
34    '任意とする', '自由とする', 'してよい',
35    'を認める', 'でもよい', '可とする',
36    '許可される', '場合はこの限り', '承認を得た',
37    '差し支えない', 'してもかまわない',
38 ]

```

```

39
40 A_PATS = [
41     '高校生らしい', '高校生として', '生徒として',
42     '華美', '節度', '品位',
43     'ふさわしい', '清潔感', '端正',
44     '適切', '生徒らしい', 'にふさわしい',
45     'にふさわしくない', '高校生らしく', '節度ある',
46     '相応しい', '相応しくない', '社会通念',
47     '良識', '見苦しくない',
48 ]
49
50 def annotate(sentence):
51     for pat in OTHER_PATS:
52         if pat in sentence:
53             return 'OTHER'
54     for pat in P_PATS:
55         if pat in sentence:
56             return 'P'
57     for pat in E_PATS:
58         if pat in sentence:
59             return 'E'
60     for pat in O_PATS:
61         if pat in sentence:
62             return 'O'
63     for pat in A_PATS:
64         if pat in sentence:
65             return 'A'
66     for pat in R_PATS:
67         if pat in sentence:
68             return 'R'
69     return 'OTHER'
70
71 def count_labels(sentences):
72     count_dict = {
73         'P': 0,
74         'O': 0,
75         'R': 0,
76         'E': 0,
77         'A': 0,
78         'OTHER': 0

```

```

79     }
80     for s in sentences:
81         label = annotate(s)
82         count_dict[label] += 1
83     return pd.Series({
84         'P_count': count_dict['P'],
85         'O_count': count_dict['O'],
86         'R_count': count_dict['R'],
87         'E_count': count_dict['E'],
88         'A_count': count_dict['A'],
89         'OTHER_count': count_dict['OTHER'],
90     })
91
92 label_df = df_schools['sentences'].apply(count_labels)
93 df_schools = pd.concat([df_schools, label_df], axis=1)
94
95 df_schools['scored_total'] = (
96     df_schools['P_count'] + df_schools['O_count'] +
97     df_schools['R_count'] + df_schools['E_count']
98 )
99
100 totals = df_schools[['P_count', 'O_count', 'R_count',
101 'E_count', 'A_count', 'OTHER_count']].sum()
102 total_sum = totals.sum()
103 scored_sum = totals[['P_count', 'O_count', 'R_count',
104 'E_count', 'A_count', 'OTHER_count']].sum()
105
106 print('全校のラベル合計:')
107 print(totals)
108 print('各ラベルの割合n')
109 print(' (禁止) の割合:P', round(totals['P_count'] / total_sum * 100, 1), '%')
110 print(' (義務) の割合:O', round(totals['O_count'] / total_sum * 100, 1), '%')
111 print(' (推奨) の割合:R', round(totals['R_count'] / total_sum * 100, 1), '%')
112 print(' (例外) の割合:E', round(totals['E_count'] / total_sum * 100, 1), '%')
113 print(' (曖昧) の割合:A', round(totals['A_count'] / total_sum * 100, 1), '%')
114 print(' (その他) の割合OTHER: ',
115 round(totals['OTHER_count'] / total_sum * 100, 1), '%')

```

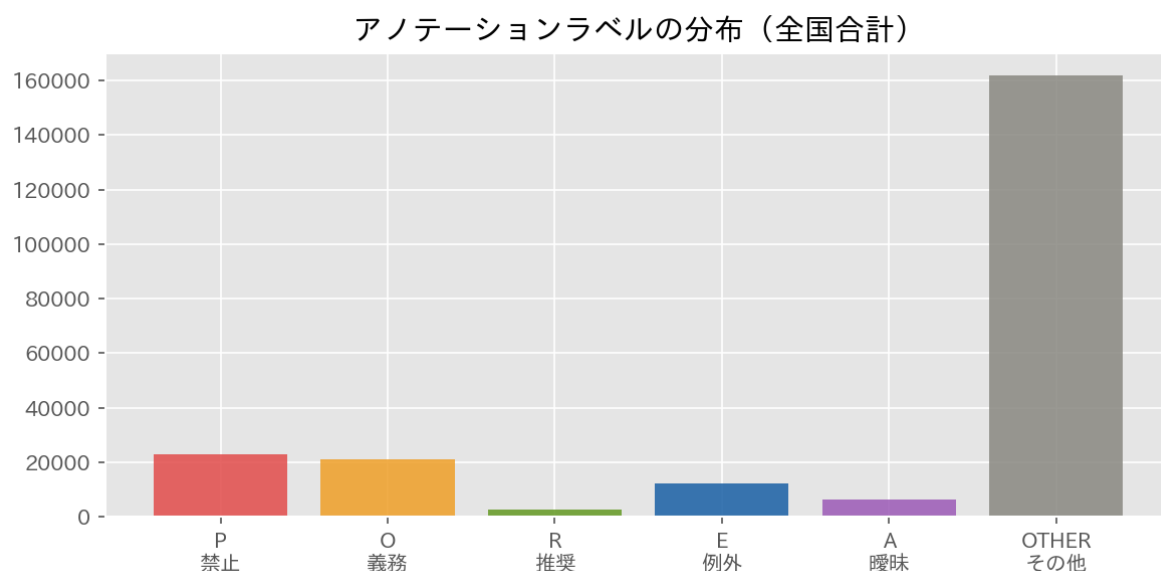
3.3.3 A ラベルの追加

辞書は当初 P・O・R・E の 4 ラベルで設計していた。しかし、校則テキストの中に「高校生らしい」「華美でない」など、明示的な禁止ではないものの解釈を学校に委ねる曖昧な規範が多数存在することが判明した。これらは生徒の行動を直接禁止するのではなく、解釈を通じて間接的に行動を方向づける。このため A ラベルを追加した。

3.3.4 ラベル分布の結果

アノテーションの結果を図 2 に示す。全条文のうち P（禁止）が 10.1%、O（義務）が 9.2%、R（推奨）が 1.3%、E（例外）が 5.4%、A（曖昧）が 2.9%、OTHER（未分類）が 71.2% を占めた。

図 2



約 7 割が OTHER に分類されたことは、当初アノテーションの失敗と捉えていた。しかし、校則を繰り返し確認する中で、この失敗自体が校則の本質を示している可能性に気づいた。校則には禁止や義務だけでなく、理念・人格形成・生活態度・教育方針といった記述が大量に含まれており、校則は単なる規則ではなく「望ましい生徒像」を構築する教育的なテキストとして機能していた。このことについては第 5 章で考察する。

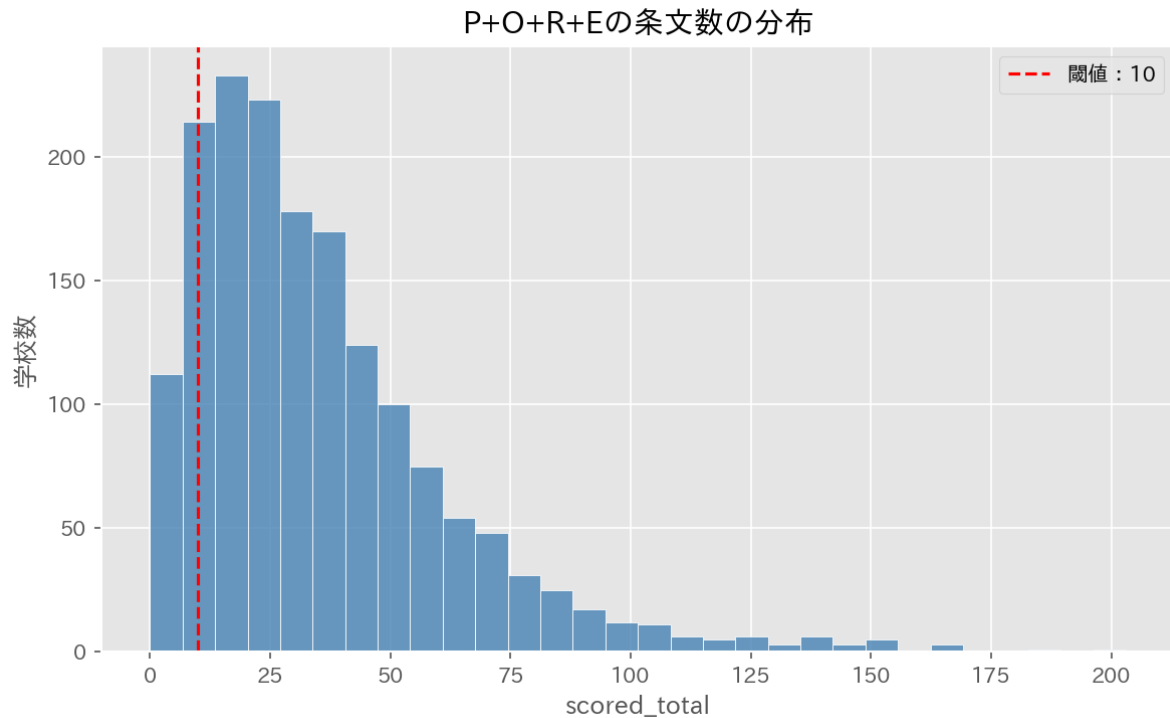
3.4 自由度スコアの算出

各学校の校則に対して、以下の式で自由度スコアを算出した。

$$\text{自由度スコア} = \frac{R \times 1 + E \times 2}{P \times 2 + O \times 1 + R \times 1 + E \times 2} \times 100 \quad (2)$$

P（禁止）と E（例外）は行動の制限・許可を最も直接的に表す表現として重み 2、O（義務）と R（推奨）は中間的な表現として重み 1 を付与した。A ラベルはスコア計算に含めず、別途集計した。これは A ラベルが禁止・許可の軸に位置付けにくい性質を持つためである。

図 3



閾値として、P+O+R+E の合計（scored_total）が 10 未満の学校はスコアを NaN として除外した。これは分母が小さすぎる場合にスコアが実態を反映しないためである（例：P=0、E=1 のみでスコア 100 となるが、これは「自由な学校」ではなく条文数の不足を意味する）閾値の妥当性は $10 \cdot 25 \cdot 50$ で比較し、分布の形状がほぼ同一であることを確認した上で、分析対象を最大化する観点から 10 を採用した（図 3）。算出された自由度スコアの分布を図 4 に示す。

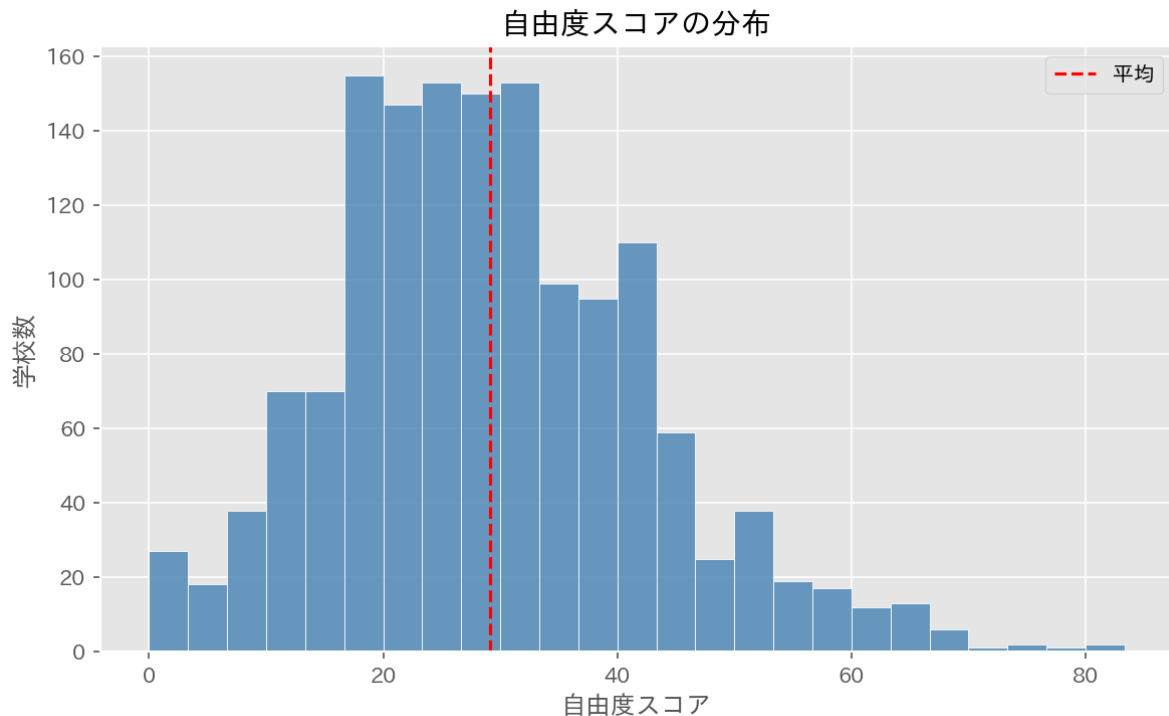
ソースコード 8 自由度スコアの算出

```

1 MIN_SCORED = 10
2 def freedom_score(row):
3     if row['scored_total'] < MIN_SCORED:
4         return np.nan
5     P, O, R, E = 2, 1, 1, 2
6     numer = row['R_count'] * R + row['E_count'] * E
7     denom = row['P_count'] * P + row['O_count'] * O +
8     row['R_count'] * R + row['E_count'] * E
9     if denom == 0:
10        return np.nan
11    return round(numer / denom * 100, 2)
12
13 df_schools['freedom_score'] = df_schools.apply(freedom_score, axis=1)

```

図 4



3.5 データ結合

全国校則一覧と高校偏差値.net では学校名の表記が異なる場合がある（例：「札幌北高等学校（全日・定時）」と「札幌北高校」）。そこで設置者名の除去、「高等学校」→「高校」への統一、括弧以降の除去を行った上で、都道府県と正規化済み学校名の 2 変数をキーに `pd.merge` で inner join を実施した。inner join を選択した理由は、両方のデータセットに存在する学校のみを分析対象とするためである。outer join にすると偏差値のない学校や校則のない学校が残り、回帰分析に支障をきたす。

ソースコード 9 データ結合

```

1 INSTALLER = [
2     '北海道立', '東京都立', '大阪府立', '京都府立', '青森県立', '岩手県立',
3     '宮城県立', '秋田県立', '山形県立', '福島県立', '茨城県立', '栃木県立',
4     '群馬県立', '埼玉県立', '千葉県立', '神奈川県立', '新潟県立', '富山県立',
5     '石川県立', '福井県立', '山梨県立', '長野県立', '静岡県立', '愛知県立',
6     '岐阜県立', '三重県立', '滋賀県立', '兵庫県立', '奈良県立', '和歌山県立',
7     '鳥取県立', '島根県立', '岡山県立', '広島県立', '山口県立', '徳島県立',
8     '香川県立', '愛媛県立', '高知県立', '福岡県立', '佐賀県立', '長崎県立',
9     '熊本県立', '大分県立', '宮崎県立', '鹿児島県立', '沖縄県立',
10    '市立', '区立', '町立', '村立', ]
11

```

```

12 def normalize_name(name):
13     name = str(name)
14     for s in INSTALLER:
15         name = name.replace(s, '')
16     name = name.replace('高等学校', '高校')
17     if '(' in name:
18         name = name.split('(')[0]
19     return name.strip()
20
21 df_schools['name_norm'] = df_schools['school_name'].apply(normalize_name)
22 hensachi.did['name_norm'] = hensachi.did['school_name']
23 .apply(normalize_name)
24
25 # 同一校に複数の偏差値がある場合は最大値を採用する
26 df_h_dedup = (
27     hensachi.did
28     .groupby(['prefecture', 'name_norm'], as_index=False, observed=True)
29     .agg(
30         hensachi = ('hensachi', 'max'),
31         gender = ('gender', 'first'),
32         gakka = ('gakka', 'first'),
33         did_ratio = ('did_ratio', 'first'),
34         location = ('location', 'first'), ) )
35
36 df_merged = pd.merge(
37     df_schools,
38     df_h_dedup,
39     on=['prefecture', 'name_norm'],
40     how='inner' )
41
42 print('校則データ件数:', len(df_schools))
43 print('偏差値データ件数:', len(df_h_dedup))
44 print('結合後件数:', len(df_merged))
45 print('マッチ率:', round(len(df_merged) / len(df_schools) * 100, 1), '%')

```

結合の結果、校則データ 1,666 校のうち 1,439 校がマッチした（マッチ率 86.4%）。マッチしなかった主な原因は、分校・通信制校・定時制校など高校偏差値.net に掲載のない学校の存在であった。偏差値・自由度スコア・DID 人口比率の全てが揃っている学校に絞り、最終的に 1,288 校を分析対象とした。


```

1 df = df_merged.loc[
2     df_merged['hensachi'].notnull() &
3     df_merged['freedom_score'].notnull() &
4     df_merged['did_ratio'].notnull(), :].copy()
5
6 df['hensachi'] = df['hensachi'].astype(float)
7 df['did_ratio'] = df['did_ratio'].astype(float)
8 df['is_girl'] = df['gender'].apply(lambda x: 1 if x == '女子' else 0)
9 df['is_boy'] = df['gender'].apply(lambda x: 1 if x == '男子' else 0)
10
11 SENMON_WORDS = ['工業', '商業', '農業', '水産',
12 '家政', '看護', '福祉', '芸術', '体育', '理数']
13 def check_senmon(gakka):
14     for s in SENMON_WORDS:
15         if s in str(gakka):
16             return 1
17     return 0
18 df['is_senmon'] = df['gakka'].apply(check_senmon)

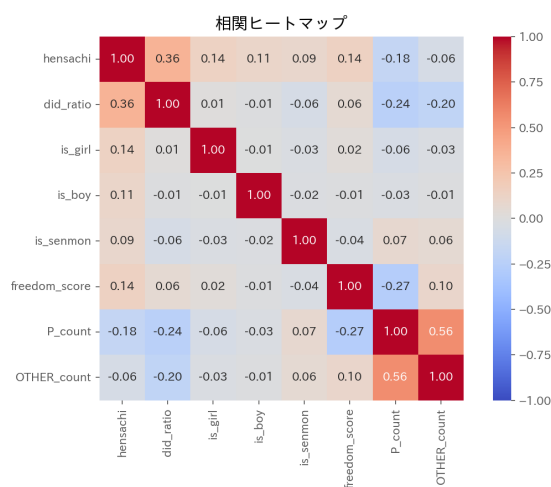
```

4 データ分析

4.1 相関ヒートマップ

分析に先立ち、主要変数間の相関関係を確認した(図 5)。偏差値と自由度スコアの間に弱い正の相関、偏差値と P_count (禁止条文数) の間に弱い負の相関が観察された。

図 5



4.2 記述統計

最終分析対象 1,288 校の基本統計量を表 2 に示す。

表 2 分析対象の基本統計量 ($n = 1,288$)

変数	平均	標準偏差	最小	最大
偏差値	48.77	9.07	36	75
DID 人口比率 (%)	50.84	37.62	0	100
自由度スコア	29.35	13.20	0	83.33
禁止条文数	14.86	9.94	0	87

4.3 重回帰分析 (RQ1: 自由度スコア)

偏差値が都市度・学校属性を統制した上でも自由度スコアと有意な関係を持つかを検証するため、以下のモデルで OLS 推定を行った。

$$\text{自由度スコア} = \beta_0 + \beta_1 \text{偏差値} + \beta_2 \text{DID 人口比率} + \beta_3 \text{女子校} + \beta_4 \text{男子校} + \beta_5 \text{専門高校} + \varepsilon \quad (3)$$

結果を表 3 に示す。

表 3 重回帰分析: 自由度スコア (OLS, $n = 1,288$)

変数	β	SE	t	p
定数項	18.944	2.034	9.315	< 0.001
偏差値	0.216	0.044	4.902	< 0.001
DID 人口比率	0.001	0.010	0.114	0.909
女子校	0.300	3.834	0.078	0.938
男子校	-5.856	6.597	-0.888	0.375
専門高校	-2.256	1.306	-1.728	0.084
$R^2 = 0.023$, 自由度調整済 $R^2 = 0.020$, $F = 6.157$ ($p < 0.001$)				

偏差値は $\beta = 0.216$ ($p < 0.001$) で統計的に有意であった。すなわち、都市度や学校属性を統制した上でも、偏差値が 1 ポイント高い学校は自由度スコアが平均 0.216 ポイント高いという関係が確認された。一方、DID 人口比率・女子校・男子校・専門高校はいずれも有意ではなかった。ただし、決定係数 $R^2 = 0.023$ は極めて低く、偏差値を含む本モデルの説明変数群は自由度スコアの分散の 2.3% しか説明できていない。多重共線性の確認として VIF (分散拡大係数) を算出した結果、全変数が 1.21 以下であり、多重共線性は問題とならなかった。

4.4 重回帰分析（禁止条文数）

自由度スコアは複数のラベルを合成した指標であるため、より解釈が明確な禁止条文数（P_count）を従属変数とした同一モデルでも推定を行った。結果を表 4 に示す。

表 4 重回帰分析：禁止条文数（OLS, $n = 1,288$ ）

変数	β	SE	t	p
定数項	22.942	1.491	15.388	< 0.001
偏差値	-0.115	0.032	-3.548	< 0.001
DID 人口比率	-0.052	0.008	-6.798	< 0.001
女子校	-4.009	2.811	-1.426	0.154
男子校	-3.545	4.836	-0.733	0.464
専門高校	2.451	0.957	2.560	0.011
$R^2 = 0.074$, 自由度調整済 $R^2 = 0.070$, $F = 20.40$ ($p < 0.001$)				

禁止条文数モデルでは、偏差値（ $\beta = -0.115$, $p < 0.001$ ）と DID 比率（ $\beta = -0.052$, $p < 0.001$ ）がともに有意な負の係数を示した。偏差値が高い学校ほど、また都市度が高い学校ほど、禁止条文の数が少ない。決定係数は $R^2 = 0.074$ であり、自由度スコアモデル（ $R^2 = 0.023$ ）の約 3.2 倍であった。禁止条文数の方が本モデルの説明変数群でよりよく説明できている。

4.5 偏差値帯別分析（RQ2）

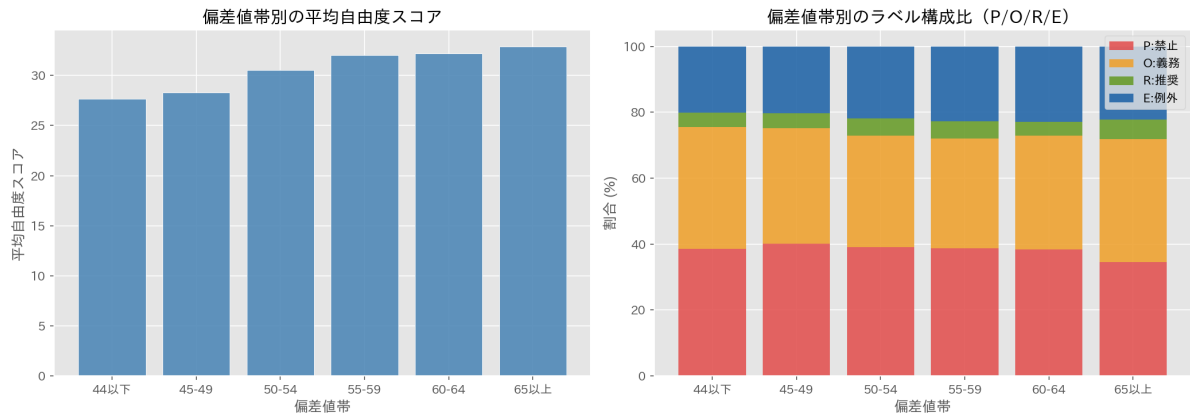
偏差値を 6 段階に区分し、各帯の平均自由度スコアとラベル構成比を集計した（表 5）。可視化の結果を図 6 に示す。

表 5 偏差値帯別の集計

偏差値帯	n	自由度スコア平均	P 平均	DID 人口比率平均
44 以下	542	27.67	16.14	36.63
45-49	250	28.27	15.69	52.82
50-54	170	30.52	15.14	61.30
55-59	126	32.03	12.68	61.74
60-64	96	32.20	12.83	68.85
65 以上	104	32.90	10.18	73.15

偏差値が上がるにつれて自由度スコアが単調に増加し（27.67→32.90）、禁止条文数が単調に減少する（16.14→10.18）傾向が確認された。一方、65 以上の層では DID 人口比率の平均が 73.15 と高く、都市部の学校に偏っていることも読み取れる。

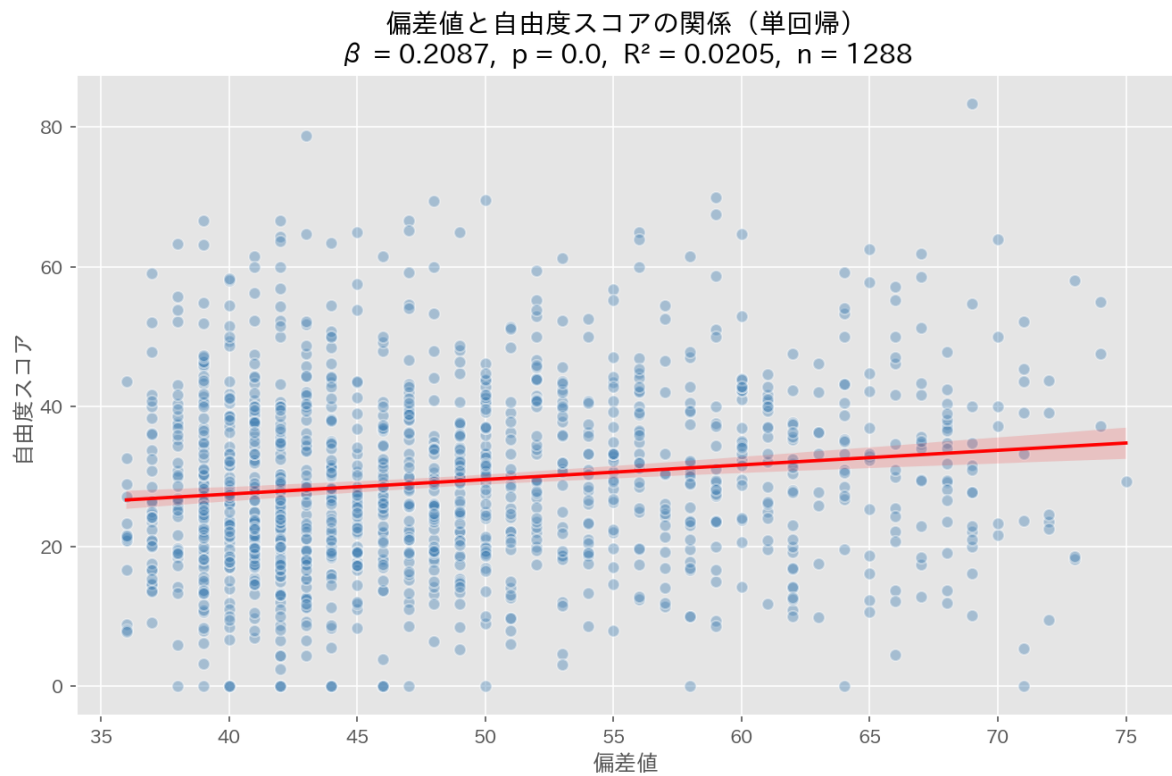
図 6



4.6 散布図

偏差値と自由度スコアの関係を散布図で可視化した (図 7.)。回帰直線の傾きは正であるが、分散が大きく、 R^2 の低さを視覚的にも確認できる。

図 7 Enter Caption



5 考察と結論

5.1 主要な知見

本研究で得られた主な知見は以下の3点である。

1. **偏差値と自由度スコアの有意な正の関係**：重回帰分析の結果、偏差値は都市度・学校属性を統制した上でも自由度スコアと有意な正の関係を持つことが確認された ($\beta = 0.216, p < 0.001$)。先行研究（大森, 2023）の示唆した傾向が、32校から1,288校に拡大した分析でも再現された。
2. **禁止条文数でDID人口比率が有意に**：自由度スコアではDID人口比率が有意でなかったが、禁止条文数を従属変数にすると有意な負の効果が現れた ($\beta = -0.052, p < 0.001$)。都市部の学校ほど禁止条文が少ないという関係は、自由度スコアのような合成指標では検出されにくい、個別ラベルでは明瞭に現れる。
3. **偏差値帯別の単調な傾向**：偏差値帯別分析では、偏差値が上がるにつれて自由度スコアが単調に増加し、禁止条文数が単調に減少する傾向が確認された。RQ2に対する回答として、偏差値が低い層では禁止・義務的な表現の比率が高く、高い層では相対的に例外・推奨的な表現が増えることが示された。

5.2 OTHERの解釈

全条文の71.2%がOTHERに分類された。当初これをアノテーションの限界と捉えていたが、校則テキストを丁寧に読み返すと、OTHERに分類された文の多くは「本校生徒は豊かな人間性を目指す」「社会に貢献する人材を育成する」といった理念・人格形成に関する記述であった。校則は禁止条項の集合として捉えられがちだが、実際にはその大部分が「望ましい生徒像」の記述で占められている。禁止や義務は校則の約20%に過ぎず、残りの80%は教育的テキストとして「こうありたい生徒」を構築する機能を担っていた。

5.3 規定されない自由

本研究の自由度スコアは、校則テキストに明記された規範表現の比率から算出されている。しかし、本当に自由な学校は「そもそも規定を設ける必要がない」ために校則テキスト自体が短く、分析に必要な条文数すら確保できない可能性がある。実際、scored_totalのフィルタ(< 10)で除外された学校には、校則が極めて短い「本当に自由な」学校が含まれているかもしれない。つまり、**自由はテキストとして現れた瞬間にしか観測できない**。これは本研究の構造的限界であり、テキスト分析一般に内在する問題である。

5.4 先行研究との比較

先行研究（大森, 2023）との比較を表6に示す。

表 6 先行研究との比較

項目	先行研究（大森, 2023）	本研究
分析対象	32 校	1,288 校
データ収集	手作業	Web スクレイピング
校則の分類	手動・主観的	MeCab + 自動アノテーション
分析手法	相関分析（Excel）	重回帰分析（statsmodels）
統制変数	なし	DID 人口比率・男女別・学科

サンプル数を約 40 倍に拡大し、統制変数を導入したにもかかわらず偏差値の有意な効果が再現されたことは、元の仮説の頑健性を支持するものである。

5.5 限界と今後の課題

1. **決定係数の低さ**： $R^2 = 0.023$ は極めて低く、校則の自由度を規定する主要因は本モデルの変数群には含まれていない可能性が高い。校風・歴史・校長の方針・地域の保護者文化など、数値化が困難な要因が存在すると考えられる。
2. **因果関係の不在**：本研究は横断的な相関分析であり、偏差値が校則の自由度を「引き起こす」という因果関係は主張できない。
3. **偏差値データの信頼性**：使用した偏差値は「全国の塾関係者から集めたデータを元に算出」されたものであり、公式データではない。一切保証しないと注記されている点は限界として認識すべきである。
4. **校則データの時点と網羅性**：全国校則一覧の校則データは 2021 年度時点のものが中心であり、最新のものと内容は異なる可能性がある。また全国 3,300 校のうち約半数の掲載にとどまっており、掲載校に偏りがある可能性がある。
5. **自動アノテーションの精度**：パターンマッチングによる自動分類では、文脈依存的な表現（「～してもよいが、教師の許可を得ること」など）の判定精度に限界がある。機械学習の導入が今後の課題である。

5.6 結論

本研究は、先行研究で示唆された「偏差値が高い学校ほど校則が緩やかである」という傾向を、全国 1,288 校の公立高校を対象とした重回帰分析で統計的に検証した。偏差値は都市度・学校属性を統制した上でも自由度スコアと有意な正の関係を持っていた。一方で、決定係数は 0.023 と低く、校則の自由度を規定する主要因は本研究のモデルでは捉えきれしていない。

校則テキストの 71% が OTHER に分類されたことは、校則が禁止条項の集合ではなく「望ましい生徒像」を構築する教育的テキストであるという発見があった。校則問題を「厳しいか緩いか」という二項対立で捉えるのではなく、テキストの多層的な機能を読み解く視点が求められる。

参考文献

- [1] 大森一輝(2023).「偏差値から考える『校則問題』」. 熊本学園大学付属高等学校 1 年次探究ポスター.https://drive.google.com/file/d/1czkCahIZV7xIQBX6ukSq-NBpF7tu1_Zi/view?usp=sharing
- [2] 全国校則一覧 (kousoku.org) . NPO 法人 Change of Perspective. <https://www.kousoku.org/> (最終閲覧日: 2026 年 5 月 30 日) .
- [3] 高校偏差値.net. <https://xn--swqwd788bm2jy17d.net/> (最終閲覧日: 2026 年 5 月 30 日) .
- [4] e-Stat (政府統計の総合窓口) . 総務省統計局. <https://www.e-stat.go.jp/> (最終閲覧日: 2026 年 5 月 30 日) .
- [5] 補 足 資 料.<https://github.com/kazuki-ok1201/kousoku-analysis/blob/main/R04%20%E6%8E%A2%E7%A9%B6%E2%85%A1%E3%83%9D%E3%82%B9%E3%82%BF%E3%83%BC.pdf>